

BAS Cycling: Business Intelligence Fallstudie - Aufgabe 3

Erstellt von: Amanullah Enayat

2026-07-11

Ziel der Analyse

Ziel dieser Analyse ist es, mithilfe einer Warenkorbanalyse (Market Basket Analysis) häufig gemeinsam gekaufte Produktgruppen der BAS Cycling zu identifizieren. Auf Grundlage der ermittelten Assoziationsregeln sollen geeignete Black-Friday-Angebote entwickelt sowie Empfehlungen für Cross-Selling und personalisierte Marketingmaßnahmen abgeleitet werden. Als Data-Mining-Verfahren wird hierzu der Apriori-Algorithmus eingesetzt.

Schritt 1: Technisches Setup in RStudio

Für unsere Analyse installieren wir das Paket `readxl`. Dieses Paket ermöglicht das Einlesen von Excel-Dateien (.xls und .xlsx) in R.

- Paket : Der erste technische Schritt im KDD-Prozess (Knowledge Discovery in Databases) ist der Import der Daten. Mit der folgenden Funktion laden wir die speziell für diese Aufgabe bereitgestellten Transaktionsdaten. Wir installieren zunächst einmal `install.packages("readxl")` über das Konsol.
- Laden der Bibliothek: Die Funktion `library()` lädt ein bereits installiertes Paket in die aktuelle R-Sitzung. Erst danach stehen dessen Funktionen – beispielsweise `read_excel()` – zur Verfügung.
- Im Gegensatz dazu ist `split()` eine Funktion, die bereits fest in die Programmiersprache R integriert ist und kein extra Laden eines Pakets erfordert (Conversation History).
- Um sicherzustellen, dass die in unserem Projekt verwendeten Pakete und deren exakte Versionen gespeichert und für die Zukunft festgehalten werden, installieren wir das Paket `renv` über die Console. Nach der Installation führen wir das Befehl `renv::init()` aus, um es in unserem Projekt integrieren.

Schritt 2: Datenimport

Im ersten Schritt des KDD-Prozesses werden die bereitgestellten Transaktionsdaten importiert.

- Die Funktion `read_excel()` liest die Excel-Datei ein und durch Zuweisung `meine_daten <-` werden die Daten dauerhaft in der Variable `meine_daten` gespeichert und können in den nächsten Analyseschritten weiterverwendet werden.
- Kommentare: Da unsere Datei in einem Unterordner namens „data“ liegt, nutzen wir einen relativen Pfad. Dabei schreiben wir den Ordnernamen, gefolgt von einem Vorwärts-Schrägstrich (/) und dem Dateinamen in Anführungszeichen. `read_excel("data/Transaktionen_Apriori.xlsx")`

Wichtiger Hinweis: Wenn wir die Funktion `read_excel("data/Transaktionen_Apriori.xlsx")` ohne Zuweisung ausführen, wird das Ergebnis nur in der Konsole angezeigt und nicht für spätere Schritte (wie die Gruppierung) gespeichert (Conversation History). Das andere Problem ist das die Daten werden nicht dauerhaft gespeichert. Sobald die Ausgabe über den Bildschirm gelaufen ist, sind sie für R wieder "vergessen". Deswegen führen wir mit Zuweisung "`<-`", damit die Daten in unserem Environment-Fenster (oben rechts in RStudio) angezieht zu bekommen.

```
meine_daten <- read_excel("data/Transaktionen_Apriori.xlsx")
```

Schritt 3: Vorbereitung der Daten (Transformation)

Da der Apriori-Algorithmus keine gewöhnliche Tabelle, sondern Transaktionen (Warenkörbe) erwartet, müssen die Daten zunächst transformiert werden.

- 3.1: Gruppierung der Daten (Transformation) Die Ausgangsdaten enthalten für jedes gekaufte Produkt eine eigene Zeile. Für die Warenkorbanalyse müssen jedoch alle Produkte, die zu derselben Bestellungsnummer gehören, zu einem gemeinsamen Warenkorb zusammengefasst werden. Hierfür wird die integrierte R-Funktion `split()` verwendet. Sie gruppiert die Produktgruppen anhand der Bestellnummer und erstellt daraus eine Liste von Warenkörben.

Erläuterung: `produkte` enthält alle Produktgruppen, `bestellungen` enthält die zugehörigen Bestellnummern. `split()` gruppiert sämtliche Produkte anhand ihrer Bestellnummer.

- Wir speichern die Bestandteile vorher in Variablen und dann gruppieren Produkte nach Bestellungen. Hier `trans_liste` ist eine Liste, aber noch kein Transaktionsobjekt.

```
produkte <- meine_daten$ProduktGroupName
bestellungen <- meine_daten$Bestellungsnummer
trans_liste <- split(produkte, bestellungen)
```

- 3.2 : Umwandlung in ein Transaktionsobjekt: Im nächsten Schritt wird die erzeugte Liste in ein Transaktionsobjekt umgewandelt. Hierzu wird das Paket `arules` verwendet. Dieses stellt die Daten in einem für den Apriori-Algorithmus geeigneten Format bereit, sodass die Transaktionen effizient verarbeitet und Assoziationsregeln ermittelt werden können.
- 3.2.2: wir installieren einmal `install.packages("arules")` über das Konsol
- 3.2.3: wir laden das Paket `library(arules)` in RStudio.
- 3.2.4: wir installieren einmal `install.packages("arulesViz")` über das Konsol für die Visualisierung der Regeln.
- 3.2.5: wir laden das Paket `library(arulesViz)` in RStudio.

```
library(arules)
```

```
## Lade nötiges Paket: Matrix
```

```
##
```

```
## Attache Paket: 'arules'
```

```
## Die folgenden Objekte sind maskiert von 'package:base':
##
##      abbreviate, write
```

```
library(arulesViz)
```

- 3.3: Transaktionsobjekt erzeugen: Das Paket `arules` erzeugt ein Transaktionsobjekt (`transactions`), das intern als spärliche Inzidenzmatrix (sparse incidence matrix) gespeichert wird. Dieses Format ermöglicht eine effiziente Verarbeitung durch den Apriori-Algorithmus.

```
warenkorb_trans <- as(trans_liste, "transactions")
```

- Kurze Überprüfung der Ergebnisse:

```
summary(warenkorb_trans)
```

```
## transactions as itemMatrix in sparse format with
## 10207 rows (elements/itemsets/transactions) and
## 38 columns (items) and a density of 0.08278374
##
## most frequent items:
##      Schutzblech      Mountainbike      Fahrradtasche      Schuhe E-Mountainbike
##           3805           2756           2375           2018           1940
##      (Other)
##           19215
##
## element (itemset/transaction) length distribution:
## sizes
##      1      2      3      4      5
## 800 2581 2668 2647 1511
##
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 1.000   2.000   3.000   3.146   4.000   5.000
##
## includes extended item information - examples:
##      labels
## 1 Accessoires
## 2          BMX
## 3          Brille
##
## includes extended transaction information - examples:
##      transactionID
## 1           41112
## 2           41113
## 3           41114
```

Schritt 4: Durchführung der Warenkorbanalyse

Nachdem die Daten erfolgreich in das Transaktionsformat überführt wurden, wenden wir nun den Apriori-Algorithmus an. Das Ziel ist es, logische Abhängigkeiten zwischen Produktgruppen zu erkennen, die häufig gemeinsam gekauft werden (Bundle-Angebote).

4.1: Begründung der gewählten Parameter

Für die Durchführung der Warenkorbanalyse wurden ein Mindestsupport von 5 % und eine Mindestkonfidenz von 30 % festgelegt.

- Mindestsupport (0,05): Eine Produktkombination muss in mindestens 5 % aller Transaktionen vorkommen, damit sie als relevant betrachtet wird. Dadurch werden sehr seltene Kombinationen ausgeschlossen, die für die Analyse meist wenig Aussagekraft besitzen.
- Mindestkonfidenz (0,30): Eine Regel wird nur berücksichtigt, wenn sie mit einer Wahrscheinlichkeit von mindestens 30 % eintritt. Das bedeutet beispielsweise: Wird Produkt A gekauft, soll in mindestens 30 % der Fälle auch Produkt B gemeinsam gekauft werden.

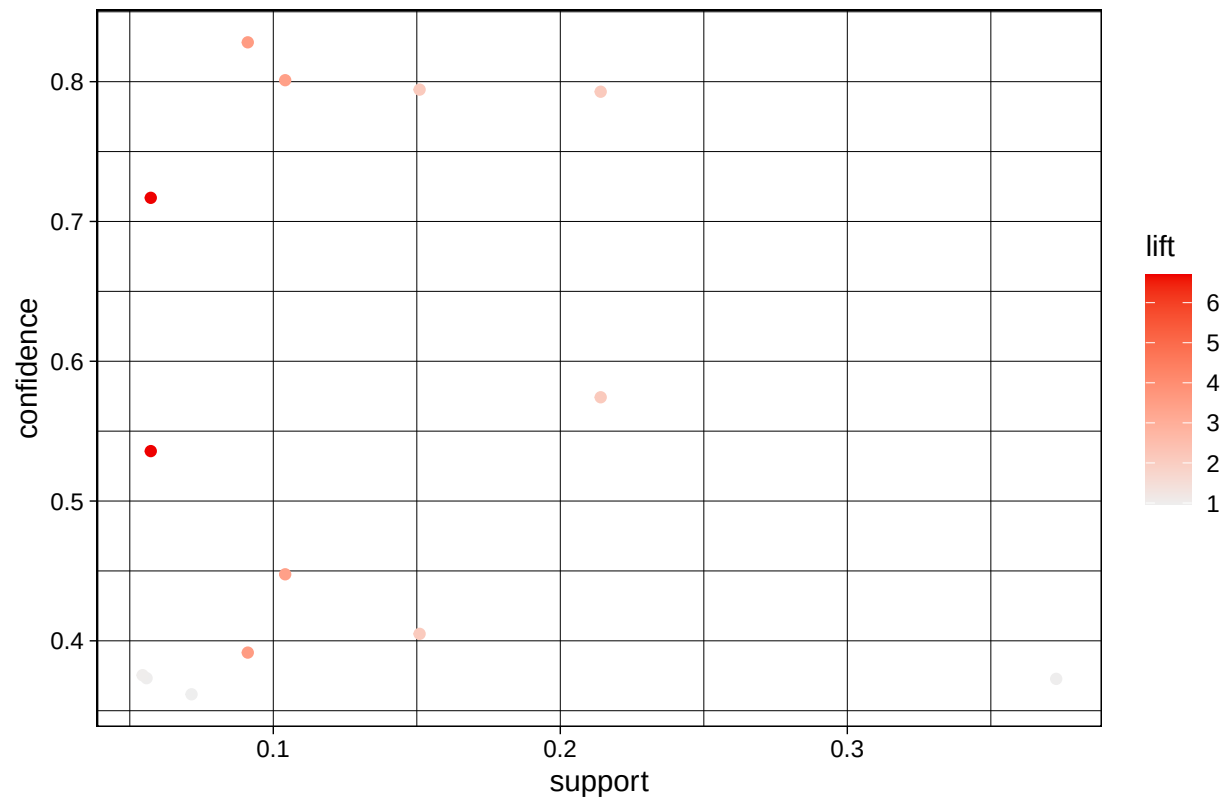
Die Werte wurden gewählt, um einen sinnvollen Kompromiss zwischen der Anzahl der gefundenen Regeln und deren Aussagekraft zu erreichen. Ein Mindestsupport von 5 % stellt sicher, dass nur häufig auftretende Produktkombinationen berücksichtigt werden. Die Mindestkonfidenz von 30 % filtert Regeln heraus, deren Vorhersagekraft zu gering ist. Dadurch bleiben ausreichend viele, aber dennoch aussagekräftige Assoziationsregeln für die weitere Analyse erhalten.

```
rules <- apriori(  
  warenkorb_trans,  
  parameter = list(  
    supp = 0.05,  
    conf = 0.30,  
    target = "rules"  
  )  
)
```

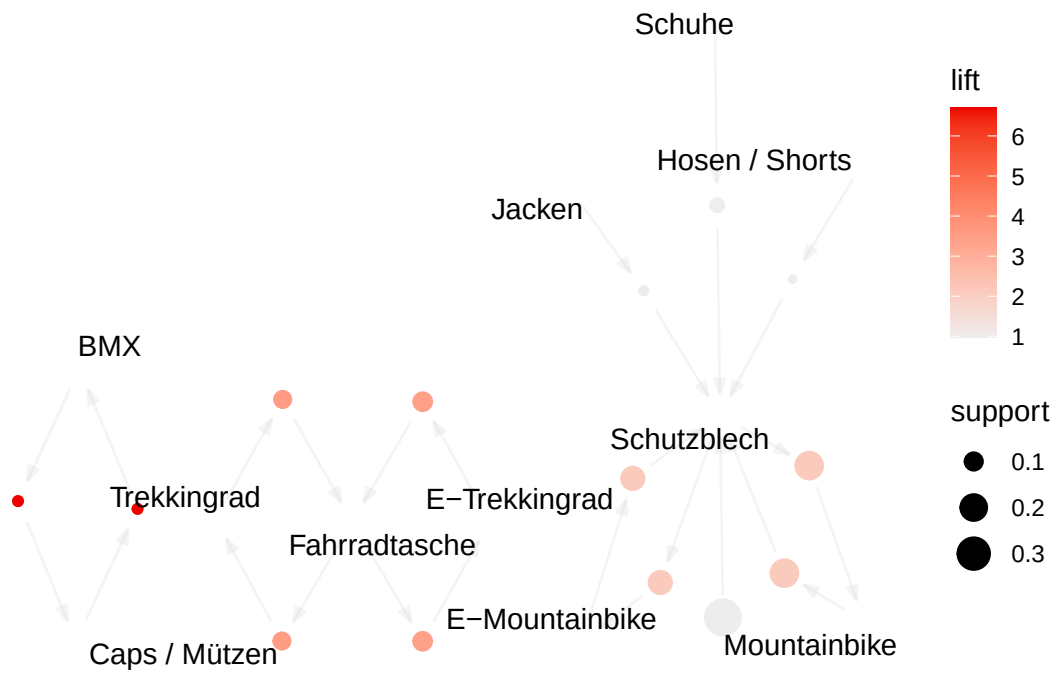
```
## Apriori  
##  
## Parameter specification:  
## confidence minval smax arem aval originalSupport maxtime support minlen  
##      0.3      0.1      1 none FALSE          TRUE      5      0.05      1  
## maxlen target ext  
##      10 rules TRUE  
##  
## Algorithmic control:  
## filter tree heap memopt load sort verbose  
##      0.1 TRUE TRUE  FALSE TRUE      2      TRUE  
##  
## Absolute minimum support count: 510  
##  
## set item appearances ...[0 item(s)] done [0.00s].  
## set transactions ...[38 item(s), 10207 transaction(s)] done [0.00s].  
## sorting and recoding items ... [19 item(s)] done [0.00s].  
## creating transaction tree ... done [0.00s].  
## checking subsets of size 1 2 3 done [0.00s].  
## writing ... [14 rule(s)] done [0.00s].  
## creating S4 object ... done [0.00s].
```

```
plot(rules, method = "scatterplot")
```

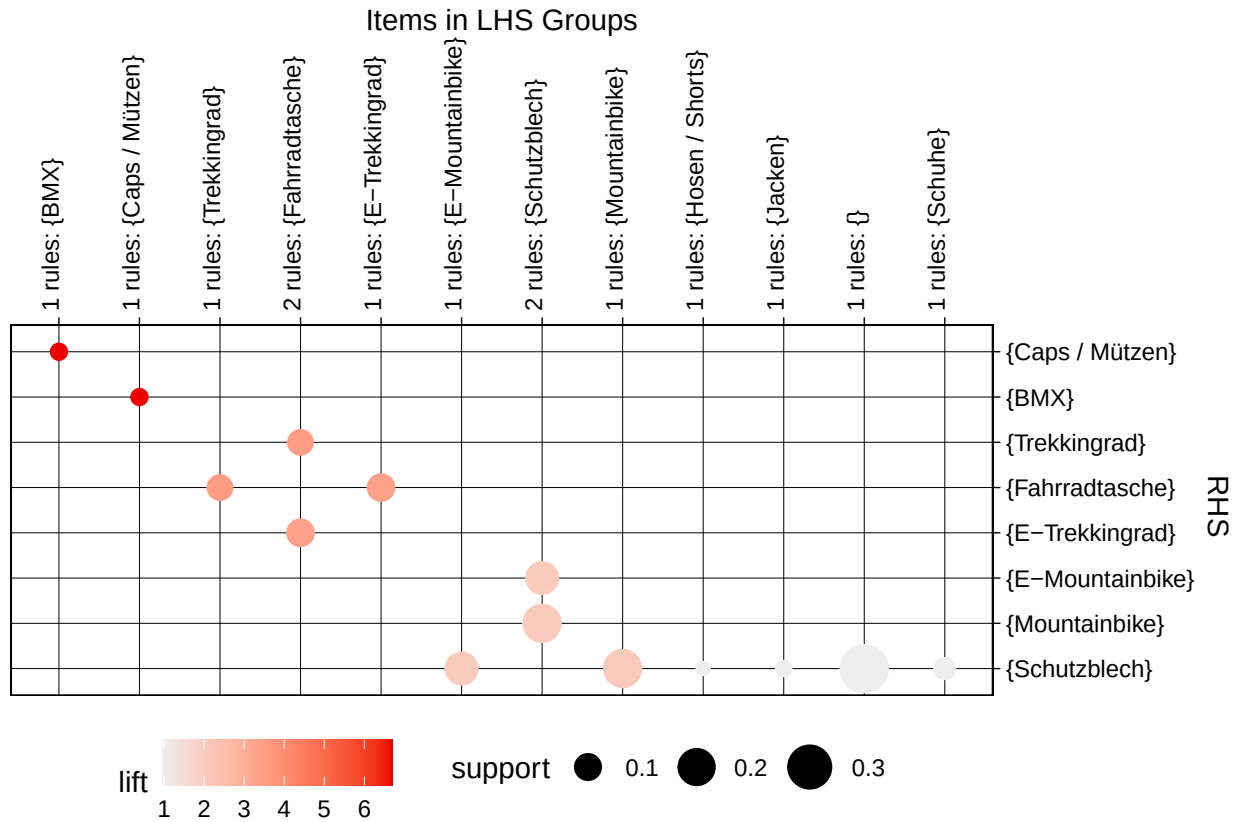
Scatter plot for 14 rules



```
plot(rules, method = "graph")
```

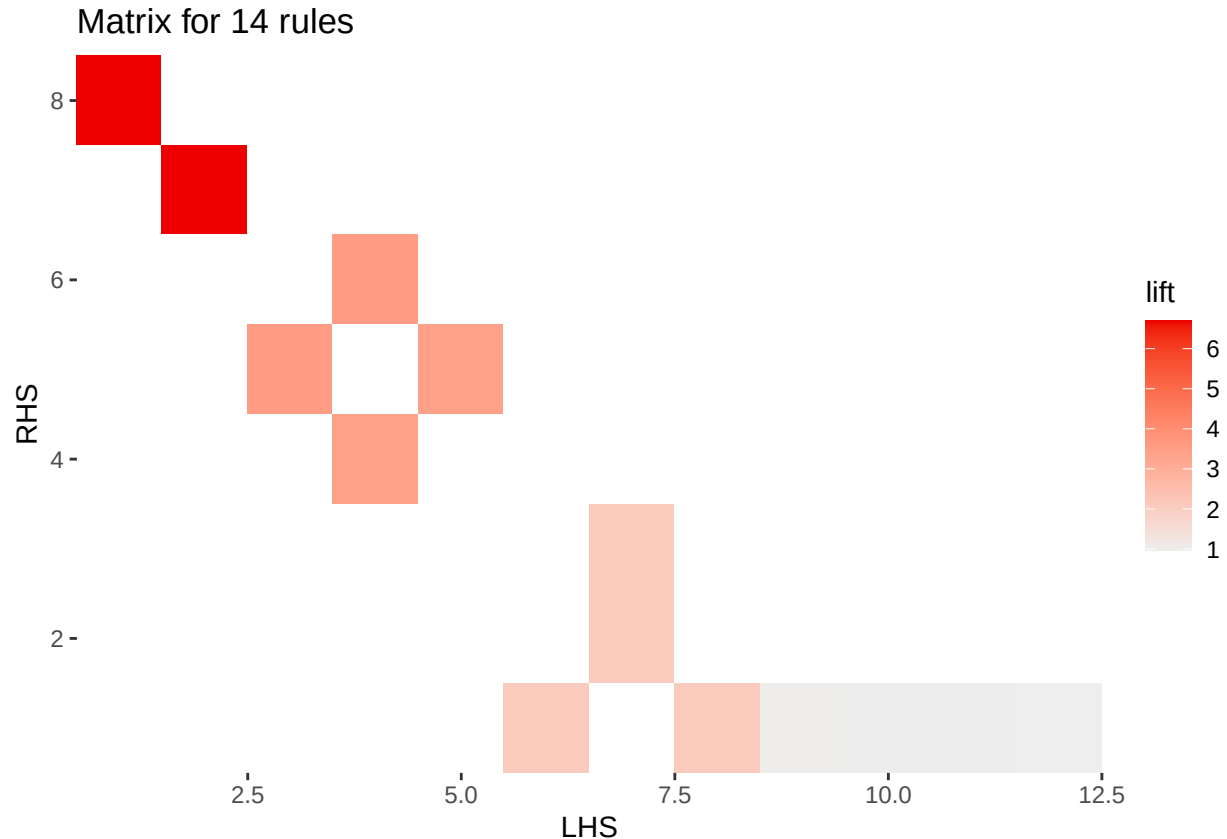


```
plot(rules, method = "grouped")
```



```
plot(rules, method = "matrix")
```

```
## Itemsets in Antecedent (LHS)
## [1] "{BMX}"           "{Caps / Mützen}" "{Trekkingrad}"   "{Fahrradtasche}"
## [5] "{E-Trekkingrad}" "{E-Mountainbike}" "{Schutzblech}"   "{Mountainbike}"
## [9] "{Hosen / Shorts}" "{Jacken}"         "{}"              "{Schuhe}"
## Itemsets in Consequent (RHS)
## [1] "{Schutzblech}"   "{Mountainbike}"   "{E-Mountainbike}" "{E-Trekkingrad}"
## [5] "{Fahrradtasche}" "{Trekkingrad}"    "{BMX}"            "{Caps / Mützen}"
```



Graph Erläuterung:

- Scatterplot Graph: Das Scatterplot zeigt jede Assoziationsregel als einzelnen Punkt. Regeln mit hohem Support und hoher Konfidenz treten häufig auf und besitzen eine hohe Aussagekraft. Ein hoher Lift weist zusätzlich darauf hin, dass der Zusammenhang zwischen den Produkten deutlich stärker ist als zufällig zu erwarten wäre. Der Scatterplot zeigt, dass mehrere Regeln sowohl einen hohen Lift als auch eine hohe Konfidenz besitzen. Besonders die Regeln $\text{BMX} \rightarrow \text{Caps/Mützen}$ sowie $\text{Trekkingrad} \rightarrow \text{Fahrradtasche}$ weisen starke Zusammenhänge auf und eignen sich daher für Bundle-Angebote.
- Netzwerkdarstellung Graph: Die Netzwerkdarstellung zeigt die Beziehungen zwischen den einzelnen Produktgruppen. Produkte mit vielen Verbindungen werden häufig gemeinsam gekauft und eignen sich besonders für Bundle-Angebote oder Cross-Selling-Maßnahmen. Die Netzwerkdarstellung verdeutlicht die engen Beziehungen zwischen Mountainbike und Schutzblech sowie Trekkingrad und Fahrradtasche. Diese Produktgruppen sollten im Online-Shop gemeinsam empfohlen werden.
- Grouped Graph: Der Grouped Graph zeigt, welche Produktgruppen gehören zusammen. Die Matrix zeigt, dass sich mehrere Produktgruppen zu wiederkehrenden Kaufmustern zusammenfassen lassen.

Interpretation der Ergebnisse: Was bedeutet der Lift?

Der Lift-Wert beschreibt die Stärke des Zusammenhangs zwischen zwei Produktgruppen. Ein Lift größer als 1 zeigt, dass zwei Produkte häufiger gemeinsam gekauft werden als zufällig zu erwarten wäre. Je höher der Lift, desto stärker ist der Zusammenhang zum Beispiel für die Erstellung von Black-Friday-Bundles.

```
rules_sorted <- sort(rules, by = "lift")
```


- Top-10-Regeln: Nachdem die Regeln im vorherigen Schritt nach ihrer Relevanz (dem Lift-Wert) sortiert wurden, lassen wir uns nun die zehn stärksten Ergebnisse anzeigen.

`head(rules_sorted, 10)`: wählt die zehn ersten Regeln der nach dem Lift sortierten Regelmenge aus.
`inspect()`: gibt die ausgewählten Regeln mit ihren wichtigsten Qualitätskennzahlen wie (support, confidence, coverage, lift und count) aus.

```
inspect(head(rules_sorted, 10))
```

```
##      lhs                rhs      support   confidence coverage
## [1] {BMX}              => {Caps / Mützen} 0.05731361 0.7169118 0.07994514
## [2] {Caps / Mützen}    => {BMX}          0.05731361 0.5357143 0.10698540
## [3] {Trekkingrad}      => {Fahrradtasche} 0.09111394 0.8281389 0.11002253
## [4] {Fahrradtasche}    => {Trekkingrad} 0.09111394 0.3915789 0.23268345
## [5] {E-Trekkingrad}    => {Fahrradtasche} 0.10414421 0.8010550 0.13000882
## [6] {Fahrradtasche}    => {E-Trekkingrad} 0.10414421 0.4475789 0.23268345
## [7] {E-Mountainbike}   => {Schutzblech} 0.15097482 0.7943299 0.19006564
## [8] {Schutzblech}      => {E-Mountainbike} 0.15097482 0.4049934 0.37278338
## [9] {Mountainbike}     => {Schutzblech} 0.21406878 0.7928157 0.27001078
## [10] {Schutzblech}     => {Mountainbike} 0.21406878 0.5742444 0.37278338
##      lift      count
## [1] 6.701024   585
## [2] 6.701024   585
## [3] 3.559080   930
## [4] 3.559080   930
## [5] 3.442681  1063
## [6] 3.442681  1063
## [7] 2.130808  1541
## [8] 2.130808  1541
## [9] 2.126746  2185
## [10] 2.126746  2185
```

Interpretation der einzelnen Regeln (Top 5)

Da mehrere Assoziationsregeln denselben Zusammenhang in umgekehrter Richtung beschreiben Zum Beispiel Regel 1: {BMX} → {Caps / Mützen} Regel 2: {Caps / Mützen} → {BMX}, wurden für die Managementbewertung nur die Regeln betrachtet, bei denen ein Hauptprodukt auf ein passendes Zubehörprodukt verweist. .

Regel 1 : BMX -> Caps / Mützen Support: 5,73 % Confidence: 71,69 % Lift: 6,70

Interpretation: Kunden, die ein BMX kaufen, erwerben mit einer Wahrscheinlichkeit von rund 72 % zusätzlich Caps oder Mützen. Der Lift von 6,70 zeigt einen sehr starken Zusammenhang, der deutlich über dem Zufall liegt.

Regel 2 : Trekkingrad -> Fahrradtasche Support: 9,11 % Confidence: 82,81 % Sehr hoch.

Interpretation: Die Mehrheit der Trekkingrad-Käufer kauft zusätzlich eine Fahrradtasche.

Regel 3 : E-Trekkingrad -> Fahrradtasche Support: 10,41 % Confidence 80,10 %

Interpretation: Auch Käufer eines E-Trekkingrads erwerben sehr häufig eine Fahrradtasche.

Regel 4 : Mountainbike -> Schutzblech Support 21,41 % (höchste Support der Regeln) Confidence 79,28 %

Interpretation: Mountainbike-Kunden kaufen häufig Schutzbleche.

Regel 5 : E-Mountainbike -> Schutzblech Support 15,09 %
Confidence 79,43 %

Interpretation: Auch E-Mountainbike-Kunden ergänzen ihren Kauf häufig um Schutzbleche.

Fazit: Die Warenkorbanalyse zeigt, dass sich mit dem Apriori-Algorithmus mehrere starke Zusammenhänge zwischen den Produktgruppen identifizieren lassen. Besonders Mountainbike-Schutzblech, Trekkingrad-Fahrradtasche und BMX-Caps/Mützen weisen hohe Lift- und Konfidenzwerte auf. Diese Erkenntnisse können genutzt werden, um attraktive Black-Friday-Angebote zu entwickeln, Cross-Selling-Potenziale auszuschöpfen und personalisierte Marketingmaßnahmen umzusetzen. Znum Beispiel:

- Personalisierte E-Mail-Nachfassaktion nach Bike-Kauf mit Rabatt auf das passende Zubehör
- Segmentierung der Kundendatenbank nach gekauften Hauptprodukten für zielgerichtete Newsletter-Ausspielung

Schritt 5: Managementempfehlungen

Black-Friday-Angebote Mountainbike + Schutzblech Trekkingrad + Fahrradtasche BMX + Caps/Mützen
E-Trekkingrad + Fahrradtasche

Cross-Selling

Beim Kauf eines Mountainbikes -> Schutzblech automatisch empfehlen Beim Kauf eines Trekkingrads -> Fahrradtasche anzeigen Beim Kauf eines BMX -> Caps/Mützen als Empfehlung einblenden Beim Kauf eines E-Trekkingrads -> Fahrradtasche anzeigen

Strategische Wahl der Analyseebene: Produktgruppen vs. Einzelprodukte Aufgrund des hochgradig diversifizierten Produktportfolios der BAS Cycling GmbH wurde die Analyse bewusst auf der Ebene der Produktgruppen (**ProduktGruppenName**) durchgeführt. Diese Entscheidung basiert auf folgenden methodischen und betriebswirtschaftlichen Erwägungen:

Sicherstellung statistischer Stabilität (Support-Schwelle): Bei einem Sortiment mit zahlreichen Varianten in Modellen, Farben und Größen wäre die Auftretenshäufigkeit jeder einzelnen spezifischen Produktkombination extrem gering. Die gewählte Support-Schwelle von 5 % würde auf Einzelproduktebene kaum erreicht werden, was zu statistisch instabilen Regeln führen würde, die nicht auf das allgemeine Kundenverhalten übertragbar sind.

Vermeidung der kombinatorischen Explosion: Bei über 10.000 Transaktionen und tausenden Einzelartikeln würde die Anzahl der möglichen Item-Kombinationen mathematisch „explodieren“, was den Rechenaufwand massiv erhöht und die Interpretation der Ergebnisse erschwert. Die Aggregation auf 38 Produktgruppen reduziert dieses „Rauschen“ und macht signifikante Muster überhaupt erst sichtbar.